VERIQTA

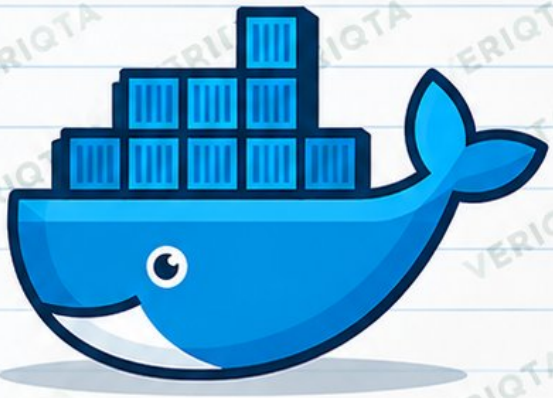
VERIQTA

The Anatomy of a Dockerfile: Every Line Explained

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r
    requirements.txt
EXPOSE 8000
CMD ["python", "app.py"]
```





- **Understand** Each Instruction
- **Real Examples** You Can Use
- **Common Mistakes** and How to Fix Them
- **Build Better,** Smaller and Safer Images
- **Real-World** Best Practices

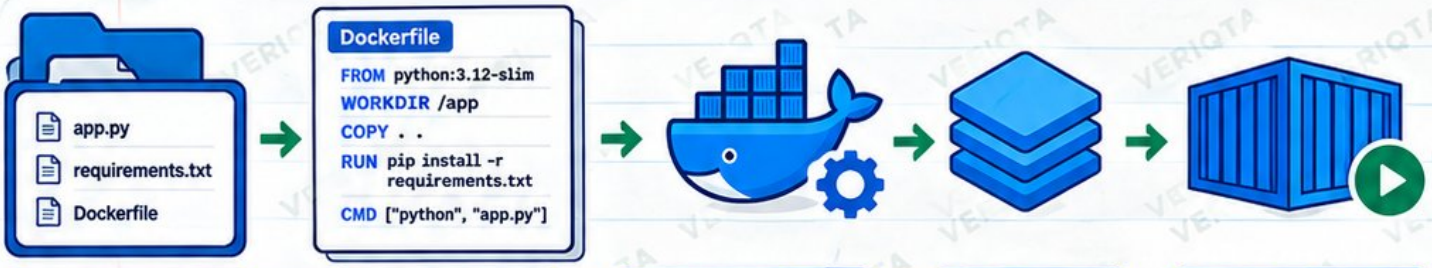
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

What Happens When Docker Reads a Dockerfile?

1. The Big Picture

Docker turns your application code and Dockerfile into a portable image, which you can run as a container.



Build Context
Your application files and Dockerfile.

Dockerfile
Tells Docker how to build the image.

Docker Builder
Reads the Dockerfile, executes each instruction, creates the image.

Image
A read-only template with your application and dependencies.

Running Container
A reimage runs as a container with your application.

2. The Flow

1. **You write a Docckerfile**
   It describes how to build the image.
2. **You provide the build context**
   Your source files, dependencies and Dockerfile.
3. **Docker builder reads the Dockerfile**
   It executes each instruction in order.
4. **An image is created**
   The result is a portable, read-only image.
5. **You run a container**
   The image becomes a running application.

Key Points

- The Dockerfile only describes how to build an image.
- It does not run your infrastructure, orchestrate containers, or manage scaling.
- Running containers, networking, storage and orchestration are handled by other tools (e.g., Docker Compose, Kubernetes, cloud platforms).

Common Mistake

Thinking the Dockerfile will start your app and handle the whole system. It only builds the image. You still need to run the container and manage it properly.

Junior Engineer Tip

Keep your Dockerfile focused on building the image. Use Docker Compose or Kubernetes for running and managing multiple containers in production.

Try This

1. Create a simple Dockerfile for a Python app.
2. Build the image: `docker build .`
3. Run a container: `docker run -p 8080:8000`

VERIQTA

Anatomy of a Real Dockerfile

1. Complete Sample Dockerfile (Numbered)

Dockerfile

```
1  # Use an official Python base image
2  FROM python:3.12-slim
3
4  # Set build argument
5  ARG APP_VERSION=1.0.0
6
7  # Set environment variables
8  ENV PYTHONDONTWRITEBYTECODE=1 \
9      PYTHONUNBUFFERED=1 \
10     APP_HOME=/app
11
12 # Set working directory
13 WORKDIR ${APP_HOME}
14
15 # Copy dependency file first (better caching)
16 COPY requirements.txt .
17
18 # Install dependencies
19 RUN pip install --no-cache-dir -r requirements.txt
20
21 # Copy application code
22 COPY . .
23
24 # Document the port the app listens on
25 EXPOSE 8000
26
27 # Set a default command to run the application
28 CMD ["python", "app.py"]
```

2. What Each Line Does

Line 1	A comment. Comments start with # and are ignored by Docker.
Line 2	FROM instruction. Chooses the base image (python:3.12-slim).
Line 5	ARG instruction. A build argument that can be passed during the build.
Lines 8–10	ENV instructions. Set environment variables that will exist in the image and container.
Line 13	WORKDIR instruction. Sets the working directory inside the image.
Line 16	COPY instruction. Copies the requirements file from your build context into the image.
Line 19	RUN instruction. Executes a command during the build (installs dependencies).
Line 22	COPY instruction. Copies the rest of your application code.
Line 25	EXPOSE instruction. Documents the port your app will listen on.
Line 28	CMD instruction. Defines the default command when the container starts.

3. Instruction Categories

Comments	# comment text (ignored by Docker)
Build arguments	ARG (values passed at build time)
Environment variables	ENV (available in image and container)
Build-time instructions	RUN, COPY, ADD (execute during build)
Image configuration	FROM, WORKDIR, EXPOSE, CMD, ENTRYPOINT, USER, LABEL, etc.

4. When Instructions Run

Run During the Build

- FROM
- ARG
- RUN
- COPY
- ADD

These instructions execute when you run docker build. They create the image and its filesystem.

Configure the Container (for when it runs)

- ENV
- WORKDIR
- EXPOSE
- CMD
- ENTRYPOINT
- USER
- LABEL
- VOLUME

These instructions do not run during build. They set configuration for the resulting image and the container that runs from it.

5. Build Stages and Execution Order

1. Docker reads the Dockerfile from top to bottom.
2. Each instruction is executed in order.
3. A new image layer may be created after certain instructions (e.g., RUN, COPY).
4. The final result is a read-only image.
5. When you run a container, Docker uses the image configuration (ENV, EXPOSE, CMD, etc.).

VERIQTA

FROM: Choosing the Foundation of Your Image

1. What is a Base Image?

- A base image is the starting point for your Docker image.
- It already contains an operating system and, optionally, a runtime (e.g., Python).
- Your Dockerfile uses the `FROM` instruction to choose this base image.

2. Syntax

```
FROM <image-name>:<tag>
```

Examples:

```
FROM python:3.12-slim
FROM ubuntu:22.04
FROM alpine:3.19
```

3. Important Concepts

- **Image name:** The repository, e.g., python, ubuntu, alpine.
- **Tag:** A specific version or variant, e.g., 3.12-slim.
- **Digest:** A unique, immutable identifier for the exact image, e.g., sha256:....
- **Official image:** Maintained by the tool vendor, e.g., python (official Python image).
- **Variants:** Different sizes and features, e.g., slim, alpine, bookworm, bullseye.

4. Why "latest" Is a Problem

- latest can change at any time.
- Your build may behave differently in the future.
- It can introduce breaking changes or security issues.
- It makes your builds non-reproducible.
- Always use a specific version tag (e.g., `3.12-slim`) or a digest (`sha256:...`).

5. Why Trusted, Maintained Images Matter

- They receive security updates.
- They are built and tested by the maintainers.
- They reduce the risk of vulnerabilities.
- They provide predictable behavior.
- Use official images or trusted sources (e.g., Docker Hub official images).

6. Popular Python Image Variants

Image	Size (approx)	Description	When to use
python:3.12	~900 MB	Full image with many packages	When you need more OS tools
python:3.12-slim	~150 MB	Smaller, essential packages only	Good default for many apps
python:3.12-alpine	~50 MB	Minimal (Alpine Linux)	Very small images, but some packages may need extra setup

7. Real Example Comparison

Image	Dockerfile Line	Build Size (approx)	Pros	Cons
Full	`FROM python:3.12`	~900 MB	Many tools included	Large size
Slim (recommended)	`FROM python:3.12-slim`	~150 MB	Smaller, good balance	Fewer OS tools
Alpine	`FROM python:3.12-alpine`	~50 MB	Very small size	Some Python packages may require extra steps

8. Using an Image Digest (Most Reproducible)

```
FROM python@sha256:3f2c...(long digest)
```

- Pins the exact image.
- Guarantees the same content every time.

Junior Engineer Tip

- Start with a specific version tag (e.g., 3.12-slim).
- Use a digest for maximum reproducibility.
- Choose the smallest image that meets your needs.

VERIQTA

WORKDIR: Controlling Where Commands Execute

1. What is WORKDIR?

- **WORKDIR** sets the current working directory inside the image.
- It tells Docker where to run subsequent commands like **RUN**, **COPY**, **ADD**, **CMD** and **ENTRYPOINT**.
- If the directory does not exist, Docker automatically creates it.

2. How WORKDIR Affects Other Instructions

After you set **WORKDIR**, all subsequent instructions use this directory as the current location.



3. Basic Examples

```
# Set working directory
WORKDIR /app

# Use relative path (new directory created automatically)
WORKDIR /app/src

# Subsequent commands run in /app/src
COPY requirements.txt .
RUN pip install -r requirements.txt
```

4. Relative vs Absolute Paths

Absolute Path

Starts from the root of the filesystem.

```
WORKDIR /app
WORKDIR /opt/myapp
```

Always goes to the exact location you specify.

Relative Path

Relative to the current working directory.

```
WORKDIR /app
WORKDIR src
```

Results in /app/src (in this example).

5. How Docker Creates the Directory

If the directory does not exist, Docker creates it for you.

```
FROM python:3.12-slim
WORKDIR /myapp
RUN ls -ld /myapp
```

```
drwxr-xr-x 2 root root 4096 Oct 15 12:00 /myapp
```

✓ /myapp did not exist, but Docker automatically created it.

6. What Happens When the Working Directory Is Wrong?

If you set the wrong working directory or forget to set it, you may get missing-file errors.

```
FROM python:3.12-slim
WORKDIR /wrong
COPY app.py .
CMD ["python", "app.py"]
```

```
python: can't open file '/wrong/app.py':
[Errno 2] No such file or directory
```

✗ Docker is looking for app.py in /wrong, but the file is not there.

7. Common Use Cases

- ✓ Set a clear application directory (e.g., /app).
- ✓ Keep your application files organized.
- ✓ Make **RUN** and **COPY** commands shorter (use relative paths).
- ✓ Ensure the container starts in the right directory.
- ✓ Works well with **CMD** and **ENTRYPOINT**.

8. Key Takeaways

1. **WORKDIR** sets the current directory inside the image.
2. Docker creates the directory if it does not exist.
3. You can use absolute or relative paths.
4. All following **RUN**, **COPY**, **ADD**, **CMD** and **ENTRYPOINT** instructions use this directory.
5. A wrong working directory can cause missing-file errors.

VERIQTA

COPY: Moving Application Files Into the Image

1. What Does COPY Do?

- COPY copies files or directories from your build context (your local machine) into the Docker image.
- It is one of the most important instructions in a Dockerfile.
- It supports individual files or entire directories.
- The destination path is inside the image.

2. Syntax

```
COPY <source> <destination>
```

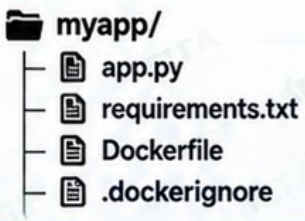
Examples:

```
COPY requirements.txt ./
COPY . .
COPY src/ /app/src/
```

3. Build Context and File Paths

- Docker only copies files that are inside the build context (the folder you run docker build from).
- You cannot COPY files from outside this folder.
- Use relative paths from the build context.

Example Project Structure (build context)



You cannot COPY files from outside the build context. Docker will give an error if you try to access files that are not in this folder.

4. Source and Destination Paths

Type	Example	What It Does
Single file	COPY requirements.txt ./	Copies the file to the current directory in the image.
Entire folder	COPY . .	Copies all files and folders to the current directory.
To a specific folder	COPY src/ /app/src/	Copies the src folder to /app/src/.
Rename file	COPY app.py /app/main.py	Copies and renames the file.

5. Practical Examples

1. Copy only the requirements file (better caching)

```
# copy requirements first
COPY requirements.txt ./
```

2. Install dependencies

```
RUN pip install --no-cache-dir -r requirements.txt
```

3. Copy the rest of the application code

```
COPY . .
```

6. Why Copying Everything Too Early Hurts Caching

- Docker builds layers in order.
- If you copy all your files first (COPY . .), any change in your code will invalidate the cache for the next steps (e.g., dependency installation).
- By copying requirements.txt first, Docker can reuse the dependency layer if your code changes but requirements.txt does not.

7. Ownership and Permissions

- By default, files are copied with root ownership.
- You can set ownership using the --chown flag.

```
COPY --chown=app:app . .
```

Useful when running the container as a non-root user.

8. Common Mistakes

- Trying to copy files from outside the build context.
- Using incorrect source or destination paths.
- Copying everything too early (causes cache misses).
- Forgetting to use .dockerignore (unnecessary files get copied).

9. Key Takeaways

- COPY copies files from your build context into the image.
- Use relative paths and keep files inside the build context.
- Copy dependency files first to improve caching.
- Use .dockerignore to exclude unnecessary files.
- Use --chown when running as a non-root user.

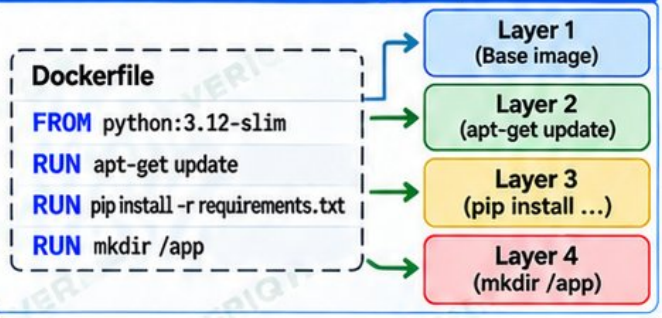
VERIQTA

RUN: Executing Commands During the Build

1. What Does RUN Do?

- RUN executes a command inside the build environment.
- It runs at build time when you build the image.
- The changes made by the command are committed to a new image layer.
- Each RUN creates a new layer in the image.
- Common uses: installing packages, creating files, setting permissions, compiling code, etc.

2. How RUN Creates Image Layers



3. Installing Python Dependencies with pip

```
# Copy requirements file first (better caching)
COPY requirements.txt ./

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

Explanation:

- Installs all Python packages listed in requirements.txt.
- --no-cache-dir reduces image size by not storing pip cache.
- This command runs during the build and the installed packages become part of the image.

4. Shell Form vs Exec Form

Shell Form (uses a shell like /bin/sh -c)

```
RUN pip install -r requirements.txt
```

- More commonly used.
- Runs the command in a shell.
- Supports shell features like &&, |, environment variable expansion, etc.

Exec Form (JSON array)

```
RUN ["pip", "install", "-r", "requirements.txt"]
```

- Does not use a shell.
- More predictable and avoids shell-related issues.
- Recommended for simple commands.

5. Real Example: Complete RUN Usage

```
FROM python:3.12-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN chmod +x start.sh && mkdir -p logs
```

6. Why Unnecessary Packages Are a Problem

- ✗ Increases image size.
- ✗ Slower build times.
- ✗ More packages to maintain.
- ✗ Higher chance of security vulnerabilities.
- ✗ Larger attack surface.
- ✗ Can introduce unwanted dependencies and conflicts.



Only install the packages you need.

7. Best Practices

- ✓ Copy requirements.txt first, then run pip install.
- ✓ Use --no-cache-dir to reduce image size.
- ✓ Combine related commands with && to reduce layers.
- ✓ Avoid installing unnecessary tools and packages.
- ✓ Clean up temporary files after installation.
- ✓ Use specific package versions when needed.

8. Common RUN Use Cases

Use Case	Example Command
Install Python packages	`RUN pip install -r requirements.txt`
Install OS packages	`RUN apt-get update && apt-get install -y curl`
Create a directory	`RUN mkdir -p /app`
Set permissions	`RUN chmod +x start.sh`
Run multiple commands	`RUN apt-get update && apt-get install -y git && rm -rf /var/lib/apt/lists/*`

VERIQTA

CMD: Defining the Default Container Command

1. What is CMD?

- **CMD** sets the default command that runs when a container starts.
- It does not run during the build. It is stored in the image metadata.
- The command is executed when you run a container from the image.
- You can override the CMD when running the container.

2. How CMD Works



- CMD is part of the image configuration.
- When you run the container, Docker uses the CMD if you do not specify another command.
- If you provide a command, your command replaces the CMD.

3. Syntax and Examples

JSON Exec Form (Recommended)

```
CMD ["python", "app.py"]
```

- Uses a JSON array.
- Does not use a shell.
- More predictable and avoids shell-related issues.
- Recommended for most use cases.

Shell Form

```
CMD python app.py
```

- Runs the command in a shell (e.g., /bin/sh -c).
- Supports shell features like &&, |, environment variable expansion, etc.
- Useful for simple commands.

4. Real Example with a Python App

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

- When you run the container, it will start your Python application by default.

▷ Build and run

```
docker build -t myapp .
docker run myapp
```

▷ Override the CMD

```
docker run myapp python -m http.server 8000
```

Your command (python -m http.server 8000) replaces the CMD ["python", "app.py"].

5. Only the Last CMD is Used

```
FROM python:3.12-slim
CMD ["python", "app.py"]
CMD ["python", "other.py"]
```

✖ Only the last CMD takes effect. In this example, ["python", "other.py"] is used.

6. CMD vs ENTRYPOINT

Feature	CMD	ENTRYPOINT
Purpose	Default command	Main executable
Can be overridden	Yes	Not easily (overridden with --entrypoint)
Typical use	Set default args or command	Set the main executable (e.g., python, nginx)
Works together	Yes (ENTRYPOINT + CMD)	Yes

7. Common Use Cases

- ✓ Start a web application: CMD ["python", "app.py"]
- ✓ Start a script: CMD ["bash", "start.sh"]
- ✓ Run a built binary: CMD ["/app/myapp"]
- ✓ Provide default arguments: CMD ["python", "manage.py", "runserver"]

8. Key Takeaways

1. CMD defines the default command for the container. It runs when the container starts, not during the build.
3. Only the last CMD in the Dockerfile is used.
4. You can override the CMD by providing a command when running the container.
5. Use the JSON exec form for better reliability.

VERIQTA

ENTRYPOINT: Controlling the Container's Executable

1. What is ENTRYPOINT?

- ENTRYPOINT sets the main executable that always runs when the container starts.
- It defines the container's primary process.
- It is different from CMD, but they can work together.
- It is useful when you always want the container to run a specific program (e.g., your application).

2. ENTRYPOINT vs CMD

Feature	ENTRYPOINT	CMD
Purpose	Main executable that always runs	Default arguments or command
Can be overridden	Requires `--entrypoint` to override	Can be overridden easily
Used together	Yes (CMD provides arguments)	Yes (provides default args to ENTRYPOINT)
Typical use	Set the main program (e.g., python, nginx)	Set default arguments (e.g., app.py, --port 8000)

3. How ENTRYPOINT and CMD Work Together

ENTRYPOINT (Executable) + CMD (Arguments) → Final Command that Runs

- Docker combines ENTRYPOINT and CMD to form the final command.
- If CMD is not specified, only ENTRYPOINT runs.
- If you override CMD, you change the arguments, not the ENTRYPOINT.

4. Exec Form vs Shell Form

Exec Form (Recommended)

```
ENTRYPOINT ["python", "app.py"]
```

- Uses a JSON array.
- Does not use a shell.
- More predictable and secure.
- Signals are handled correctly (e.g., Ctrl+C).

Shell Form

```
ENTRYPOINT python app.py
```

- Runs inside a shell (e.g., /bin/sh -c).
- Supports shell features like &&, |, environment variables.
- Can be useful for simple commands but less predictable.

5. Real Examples

Example 1: ENTRYPOINT with CMD (exec form)

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"]
```

Resulting command:

```
python app.py --port 8000
```

Example 2: ENTRYPOINT without CMD

```
ENTRYPOINT ["python", "app.py"]
```

Resulting command:

```
python app.py
```

Example 3: Using CMD only

```
CMD ["python", "app.py"]
```

Resulting command:

```
python app.py
```

6. Overriding ENTRYPOINT and CMD

Override CMD (change arguments only)

```
docker run myapp --port 9090
```

Runs: `python app.py --port 9090` (ENTRYPOINT stays the same)

Override ENTRYPOINT (change executable)

```
docker run --entrypoint bash myapp
```

Runs: `bash` (ENTRYPOINT is replaced)

Override both ENTRYPOINT and CMD

```
docker run --entrypoint python myapp -m http.server 8000
```

Runs: `python -m http.server 8000`

7. Common Mistakes and Problems

- ✗ Using the wrong executable path (file not found).
- ✗ File is not executable (permission denied).
- ✗ Incorrect JSON syntax in exec form.
- ✗ Forgetting to include CMD when arguments are needed.
- ✗ Using shell form when signal handling is important.

8. When to Use ENTRYPOINT and CMD

- ✓ Use ENTRYPOINT when you always want the container to run a specific executable.
- ✓ Use CMD to provide default arguments (e.g., port, config file).
- ✓ Use both together for a flexible and predictable setup.
- ✓ Use exec form for better reliability and security.
- ✓ Ensure the entrypoint file exists and has execute permissions.

VERIQTA

ENV: Setting Environment Variables

1. What Does ENV Do?

- ENV sets environment variables inside the image.
- The values are available to later build steps and to the running container.
- You can use these variables in RUN, CMD, ENTRYPOINT and other instructions.
- ENV is useful for application configuration, language settings, and behavior control.

2. Syntax and Examples

Set a single variable

```
ENV PYTHONDONTWRITEBYTECODE=1
```

Set multiple variables

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
```

Using variables in later instructions

```
RUN echo "Bytecode: $PYTHONDONTWRITEBYTECODE"
RUN echo "Unbuffered: $PYTHONUNBUFFERED"
```

3. Common Python Environment Variables

Variable	Value	What It Does
PYTHONDONTWRITEBYTECODE	1	Prevents Python from creating .pyc files. Keeps the image cleaner.
PYTHONUNBUFFERED	1	Disables output buffering so logs appear immediately in the container.

4. Real Example in a Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

5. Verify Environment Variables

Check inside a running container

```
docker run --rm myapp env | grep PYTHON
```

Expected output

```
PYTHONDONTWRITEBYTECODE=1
PYTHONUNBUFFERED=1
```

6. Runtime Overrides

You can override ENV values when running the container.

```
docker run --rm \
  -e PYTHONUNBUFFERED=0 \
  -e MY_ENV=production \
  myapp
```

The runtime value replaces the value set in the Dockerfile for that container only.

7. Why Not Store Credentials in ENV?

- ✗ Environment variables in a Dockerfile are stored in the image layers.
- ✗ Anyone with access to the image can view them.
- ✗ They can be exposed in image history, logs, and registries.
- ✗ Use secure methods such as Docker secrets, external secret managers, or runtime environment injection for sensitive data (e.g., API keys, database passwords).

8. Other Common Use Cases

- ✓ Set application mode: `ENV APP_ENV=production`
- ✓ Set configuration path: `ENV CONFIG_PATH=/app/config`
- ✓ Control logging: `ENV LOG_LEVEL=info`
- ✓ Set timezone: `ENV TZ=UTC`
- ✓ Configure port for app: `ENV PORT=8000`

9. Key Takeaways

1. ENV sets environment variables available during build and runtime.
2. Use it for application configuration and behavior control.
3. You can override values at runtime with `-e`.
4. Do not store sensitive information such as credentials in ENV.
5. Use secure secret management for confidential data.

VERIQTA

ARG: Passing Values Into Image Builds

1. What is ARG?

- ARG defines build-time variables that you can pass into the image build.
- The values are available during the build but are not persisted in the final image.
- You can set default values and override them with --build-arg.
- ARG is useful for things like version numbers, build configuration, and conditional logic.

2. Syntax and Example

Basic syntax

```
ARG NAME=default_value
```

Example with a default value

```
ARG APP_VERSION=1.0.0
```

Use in a build step

```
RUN echo "Building version: $APP_VERSION"
```

3. Complete Example in a Dockerfile

```
# Set build argument with a default value
ARG APP_VERSION=1.0.0

FROM python:3.12-slim

# Use the build argument
ARG APP_VERSION
RUN echo "Building version: $APP_VERSION"
LABEL org.opencontainers.image.version="$APP_VERSION"

WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

4. Building with --build-arg

Use the default value

```
docker build -t myapp .
```

Override the value

```
docker build -t myapp --build-arg APP_VERSION=2.1.0 .
```

See the value during build

```
docker build --build-arg APP_VERSION=2.1.0 .
```

5. ARG vs ENV

Feature	ARG	ENV
When it's available	Build time only	Build time and runtime
Persisted in image	No	Yes
Can be overridden	With --build-arg	With -e at runtime
Typical use	Build configuration (e.g., version, options)	Application configuration (e.g., settings, paths)
Visible in final image	No (but can appear in history)	Yes (stored in image)

6. Scope and Usage

- ARG is available from the line it iis defined, to the end of the current build stage.
- If you use multi-stage builds, you must redeclare ARG in each stage where you want to use it.
- You can use ARG in FROM to choose a base image.

Example: Using ARG in FROM

```
ARG PYTHON_VERSION=3.12
FROM python:${PYTHON_VERSION}-slim
```

7. Why ARG is Not for Secrets

- ✗ Build arguments are not secure.
- ✗ Their values can be seen in image build history (e.g., with docker history).
- ✗ They may appear in build logs, CI/CD logs, and registries.
- ✗ Anyone with access to the image or its build metadata can potentially see the values.
- ✗ Use Docker secrets, external secret managers, or runtime environment variables for sensitive information (e.g., API keys, database passwords).

8. Where ARG Values Can Be Exposed

- In image build history:

```
docker history myapp
```

- In build logs (CI/CD systems).
- In Dockerfile if committed to a public repository.
- In intermediate image layers (depending on how it is used).
- Therefore, do not pass credentials, tokens, or any sensitive data using ARG.

9. Key Takeaways

1. ARG passes values into the build.
2. It is available during the build but not persisted in the final image.
3. You can set default values and override them with --build-arg.
4. ARG and ENV serve different purposes.
5. Do not use ARG for secrets or sensitive information.

VERIQTA

EXPOSE: Documenting Application Ports

1. What is a Container Port?

- A **container port** is the port inside the container where your application listens for connections.
- Your application (e.g., a Python web server) listens on a specific port, such as **8000**.
- Other containers or the host can connect to this port if it is published.
- The port number is just a number that identifies a network endpoint for communication.

2. What Does EXPOSE Do?

- **EXPOSE** documents the port that the application inside the container listens on.
- It is **metadata** in the image, **not** a command to publish the port.
- It helps people understand which ports the container is expected to use.
- You can expose one or more ports.
- It does not make the application reachable by itself.

3. Syntax and Examples

Single port

```
EXPOSE 8000
```

Exposes port 8000 (in image metadata).

Multiple ports

```
EXPOSE 8000 8080
```

Exposes multiple ports.

With protocol (optional)

```
EXPOSE 8000/tcp
```

TCP is the default. You can also use /udp.

4. Real Example: Python Web Application

Dockerfile

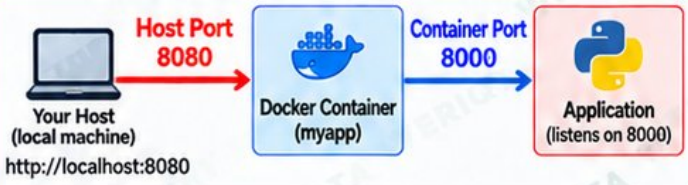
```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "app.py"]
```

Run the container and publish the port

```
docker run -p 8080:8000 myapp
```

- The application inside the container listens on port 8000 (EXPOSE 8000).
- The `-p 8080:8000` flag publishes container port 8000 to host port 8080.
- You can then access the application at http://localhost:8080 on your host machine.

5. How Port Publishing Works



6. EXPOSE vs Port Publishing

Feature	EXPOSE	docker run -p
Purpose	Documents the port	Publishes the port
Effect	Image metadata only	Maps container port to host port
Makes app reachable	No	Yes
Used at	Build time (Dockerfile)	Run time (docker run)
Required	No	Yes (to access from host)

7. Important Notes

- ✗ **EXPOSE** does not publish a port.
- ✗ **EXPOSE** does not make the application reachable.
- 8 Your application must be configured to listen on the specified port (e.g., 0.0.0.0:8000).
- 4 You must use **-p** or **--publish** when running the container to access it from your host machine.
- 5 **EXPOSE** is mainly for documentation and helps with tools like **docker ps** and orchestration platforms.
- 5 Even with **EXPOSE**, the application will not be reachable if it is not running or configured correctly.

8. Common Use Cases

- ✓ Web applications (e.g., Flask, Django, FastAPI) → **EXPOSE 8000**
- ✓ API services → **EXPOSE 8000**
- ✓ Database containers → **EXPOSE 5432**
- ✓ Multiple services → **EXPOSE 8000 9000**

9. Key Takeaways

1. **EXPOSE** documents the container's application port.
2. It does not publish the port or make the application reachable.
3. Use **docker run -p** to publish the port.
4. Your application must listen on the exposed port.
5. **EXPOSE** helps with documentation and container management.

VERIQTA

USER: Running Applications Without Root

1. Default User in Containers

- Containers run as the root user by default.
- The root user has full privileges inside the container.
- This can be risky if the application is compromised.
- You should run applications with a non-root user following the principle of least privilege.

2. Why Use a Non-Root User?

- ✓ Reduces security risk if the application is compromised.
- ✓ Limits damage to the container filesystem.
- ✓ Prevents accidental changes to system files.
- ✓ Follows security best practices.
- ✓ Many platforms (e.g., Kubernetes) recommend or require non-root containers.

3. Create a Dedicated Application User

```
FROM python:3.12-slim

# Create a non-root user and group
RUN groupadd -r app && \
    useradd -r -g app -d /app -s /bin/bash app

WORKDIR /app

# Copy files and set ownership
COPY --chown=app:app . .

# Switch to the non-root user
USER app

CMD ["python", "app.py"]
```

4. Explanation of Key Instructions

Instruction	What It Does
groupadd -r app	Creates a system group named app.
useradd -r -g app -d /app -s /bin/bash app	Creates a system user named app, sets home directory and shell.
COPY --chown=app:app . .	Copies files and sets ownership to the app user and group.
USER app	Switches to the non-root user for subsequent instructions and at runtime.

5. Verify the Running User

```
docker build -t myapp .
docker run --rm myapp whoami
docker run --rm myapp id
```

Expected output:

```
app
uid=1000(app) gid=1000(app)
groups=1000(app)
```

6. File Ownership and Permissions

- Files copied into the image are owned by root by default.
- If you switch to a non-root user without changing ownership, the application may not have permission to read or write files.
- Use COPY --chown=app:app or RUN chown -R app:app /app to set the correct ownership.

7. Example: Permission Error (Wrong Ownership)

```
FROM python:3.12-slim
RUN useradd -r app
WORKDIR /app
COPY . .
USER app
RUN touch test.txt
```

Build error:

```
touch: cannot touch
'test.txt':
Permission denied
```

8. Fix the Permission Error

```
FROM python:3.12-slim
RUN useradd -r app
WORKDIR /app
COPY --chown=app:app . .
USER app
RUN touch test.txt
```

Build succeeds:

```
Step ..
 ---> Running in ...
 ---> Successfully
      created ..
```

9. Key Takeaways

1. Containers run as root by default.
2. Use a non-root user to reduce security risk.
3. Set correct file ownership to avoid permission errors.
4. Follow the principle of least privilege.
5. Many platforms recommend or require non-root containers.
6. Use COPY --chown or chown to set ownership.
7. Always verify the running user.

VERIQTA

.dockerignore: Controlling the Build Context

1. What is Build Context?

- When you run docker build, Docker sends the **build context** (a set of files and directories) to the builder.
- By default, Docker sends everything in the current directory (.) except files excluded by **.dockerignore**.
- The build context is what Docker uses for instructions like **COPY** and **ADD**.
- Large or unnecessary files make the build slower and can increase image size.

2. What Does .dockerignore Do?

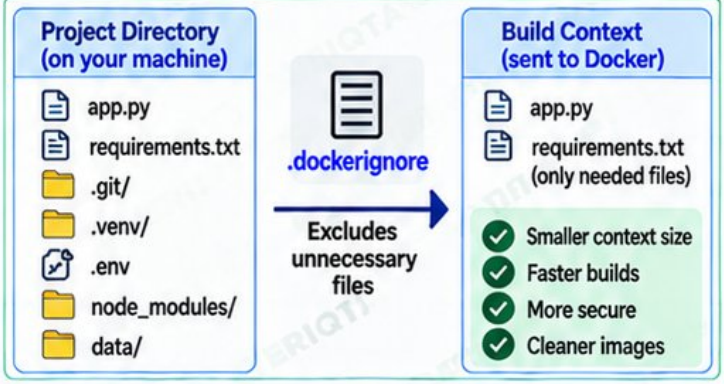
- **.dockerignore** tells Docker which files and directories to exclude from the build context.
- It works similarly to **.gitignore** but is used only by Docker.
- Excluded files are **not** sent to the builder, so they cannot be copied into the image.
- This reduces build time, decreases context size, prevents accidental inclusion of sensitive files, and improves security.

3. Example .dockerignore File

```
# Version control
.git
.gitignore
# Python files and environments
__pycache__/
*.pyc
*.pyo
*.pyd
.venv/
venv/
env/
# Dependency caches
.mypp_cache/
.pytest_cache/
.ruff_cache/
pip-wheel-metadata/
# Local and sensitive files
.env
*.env
*.key
*.pem
# OS and editor files
.DS_Store
Thumbs.db
.vscode/
.idea/
# Build and distribution files
dist/
build/
*.egg_info/
```

- Excludes Git folder and files
- Excludes Python cache and virtual environments
- Excludes dependency caches
- Excludes local secret and environment files
- Excludes OS and editor files
- Excludes build output files

4. How It Works



5. .dockerignore vs .gitignore

Feature	.dockerignore	.gitignore
Used by	Docker build process	Git version control
Purpose	Exclude files from build context	Exclude files from Git repository
Affects	What Docker sends to the builder	What Git tracks and commits
Required for Docker	Yes (recommended)	No
File contents	Can be similar but not always the same	Can be similar but not always the same

6. Verify What Files Are Sent

Build an image and check the build context size.

```
docker build -t myapp .
# Look for output like:
=> [internal] load build context
=> transferring context: 5.2MB
```

If the size is large, you may be sending unnecessary files. Use **.dockerignore** to reduce it.

7. Why Exclusions Matter

- Faster builds (less data to send).
- Smaller context size and smaller images.
- Prevents sensitive files (e.g., **.env**, keys) from being copied.
- Avoids copying local development files and caches.
- Reduces the risk of exposing private code or data.

8. Common Mistakes

- Forgetting to exclude **.git** (increases context size).
- Including **.env** or secret files.
- Not excluding virtual environments and caches.
- Assuming **.gitignore** automatically applies to Docker (it does not).
- Sending large data files, logs, or local build artifacts.

9. Key Takeaways

1. **.dockerignore** controls which files Docker sends to the builder.
2. It helps you build faster, smaller, and more secure images.
3. It is different from **.gitignore**.
4. Exclude unnecessary, sensitive, and local development files.
5. Always check the build context size when building your image.

VERIQTA

Docker Build Cache: Why Instruction Order Matters

1. What is Docker Build Cache?

- Docker caches the result of each instruction layer.
- If an instruction and its inputs do not change, Docker reuses the cached layer instead of running the instruction again.
- This makes builds faster, saves resources, and reduces unnecessary work.
- Each layer is identified by the instruction and the files it uses.

2. When is the Cache Invalidated?

- ✗ When the instruction itself changes.
- ✗ When any file used by the instruction changes.
- ✗ When files copied with COPY change.
- ✗ When the base image changes.
- ✗ When build arguments used in the instruction change.
- ✗ When earlier layers change (this affects all following layers).

3. Bad Example: Copy Everything Early

```
# Dockerfile.bad
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

Problem:

- ✗ COPY . . copies all files, including application code.
- ✗ If any source code file changes, the COPY layer changes.
- ✗ This invalidates the cache for the RUN pip install step.
- ✗ Dependencies are reinstalled even if they have not changed, making builds slower.

4. Good Example: Copy Dependency Files First

```
# Dockerfile.good
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Why this is better:

- ✓ Only requirements.txt is copied first.
- ✓ The RUN pip install layer is cached as long as requirements.txt does not change.
- ✓ When application code changes, only the later COPY . . layer is rebuilt.
- ✓ This reduces rebuild time and keeps dependency layers cached.

5. How Cache Reuse Works

FROM python:3.12-slim →	Cached (if unchanged)
WORKDIR /app →	Cached (if unchanged)
COPY requirements.txt . →	Cached (if file unchanged)
RUN pip install -r requirements.txt →	Cached (if requirements.txt unchanged)
COPY . . →	Rebuilt if source code changes
CMD ["python", "app.py"] →	Cached (if unchanged)

6. Controlled Rebuild Exercise

1. Create a simple Python app with a requirements.txt file.
2. Use the good Dockerfile (copy dependency file first).
3. Build the image:
   ```
   docker build -t myapp:1 .
   ```
4. Change only a source code file (e.g., app.py) and rebuild:
   ```
   docker build -t myapp:2 .
   ```
5. Observe that the pip install layer is reused from cache.
6. Now change requirements.txt (e.g., add a package) and rebuild:
   ```
   docker build -t myapp:3 .
   ```
7. Observe that the pip install layer is rebuilt because the dependency file changed.

7. Key Takeaways

1. Instruction order affects cache efficiency.
2. Copy dependency files before application code.
3. Unchanged files allow Docker to reuse cached layers.
4. Changing source code should not trigger dependency reinstall.
5. Use a .dockerignore file to avoid copying unnecessary files.

8. Real-World Impact

- ✓ Faster build times during development.
- ✓ Lower bandwidth and compute usage in CI/CD.
- ✓ More efficient Docker layer caching in remote registries.
- ✓ Reduced costs and fewer image rebuilds.
- ✓ A smoother development workflow.

VERIQTA

Docker Build Cache: Why Instruction Order Matters

1. What is Docker Build Cache?

- Docker caches the result of each instruction (e.g., FROM, COPY, RUN) to speed up future builds.
- If an instruction and its input files have not changed, Docker reuses the cached layer instead of running it again.
- This makes builds faster and reduces unnecessary work, especially for steps like installing dependencies.

2. When is the Cache Invalidated?

- ✗ When the instruction itself changes (e.g., different command).
- ✗ When the files used by the instruction change.
- ✗ When a previous instruction is not cached.
- ✗ When you copy the entire source code early, any code change will invalidate the cache for later steps.

3. Dockerfile with Good Instruction Order (Better Caching)

```
1  FROM python:3.12-slim
2  WORKDIR /app
3
4  # 1. Copy only dependency files first
5  COPY requirements.txt .
6  RUN pip install --no-cache-dir -r requirements.txt
7
8  # 2. Copy application code later
9  COPY . .
10 CMD ["python", "app.py"]
```

These layers change rarely, so they are usually cached.

Application code changes frequently, so only this layer rebuilds.

4. Dockerfile with Poor Instruction Order (Slower Builds)

```
1  FROM python:3.12-slim
2  WORKDIR /app
3
4  # Copy everything early
5  COPY . .
6
7  # Install dependencies after code
8  RUN pip install --no-cache-dir -r
       requirements.txt
9  CMD ["python", "app.py"]
```

- ✗ Any change in your source code will invalidate the cache for the pip install step.
- ✗ This means dependencies are reinstalled every time, making builds slower.

5. How Cache Reuse Works

FROM python:3.12-slim	✓ Cached (if unchanged)
WORKDIR /app	✓ Cached (if unchanged)
COPY requirements.txt .	✓ Cached (if requirements.txt unchanged)
RUN pip install ...	✓ Cached (if requirements.txt unchanged)
COPY . .	✗ Rebuilt (if any source code changed)
CMD ["python", "app.py"]	✓ Cached (if unchanged)

6. Effect of Different Changes

What Changed?	Good Order (Rebuilds)	Poor Order (Rebuilds)
No changes	Uses cache (very fast)	Uses cache (very fast)
Only application code (e.g., app.py)	Rebuilds only COPY . . and later layers	Rebuilds COPY . . and pip install step
requirements.txt changed	Rebuilds COPY requirements.txt and pip install step	Rebuilds COPY . . and pip install step
Base image changed (e.g., python:3.12-slim)	Rebuilds from top	Rebuilds from top

7. Controlled Rebuild Exercise

1. Build the image for the first time.
```
docker build -t myapp:cache .
```
2. Build again without any changes (should use cache).
```
docker build -t myapp:cache .
```
3. Make a small change to app.py and build again.
```
echo "# change" >> app.py
docker build -t myapp:cache .
```
4. Make a change to requirements.txt and build again.
```
echo "requests==2.32.0" >> requirements.txt
docker build -t myapp:cache .
```

8. Tips to Maximize Cache Usage

- ✓ Copy dependency files (e.g., requirements.txt, package.json) first.
- ✓ Install dependencies before copying the rest of your source code.
- ✓ Avoid copying unnecessary files (use .dockerignore).
- ✓ Keep instructions stable and avoid changing them frequently.
- ✓ Use specific base image versions (e.g., python:3.12-slim) instead of latest.

9. Key Takeaways

1. Docker caches each instruction to speed up builds.
2. Cache is invalidated when the instruction or its input files change.
3. Copying dependency files before application code reduces rebuild work.
4. Changing source code should not trigger dependency reinstallation.
5. Good instruction order leads to faster, more efficient builds.

VERIQTA

ADD: When COPY Is Not Enough

1. What is ADD?

- ADD copies files from the build context to the image.
- It has extra capabilities compared to COPY.
- It can automatically extract local tar archives.
- It can also download files from remote URLs (HTTP/HTTPS).
- Because of these extra features, ADD is more powerful but also less explicit than COPY.
- For most application files, COPY is clearer and preferred.

2. Key Capabilities of ADD

1. Copies files and directories from the build context.
2. Automatically extracts local tar archives (e.g., .tar, .tar.gz, .tgz, .tar.bz2, .tar.xz).
3. Supports remote sources (HTTP/HTTPS URLs).
4. Preserves file permissions and timestamps (similar to COPY).
5. Can be useful for fetching and extracting prebuilt tools or binaries.

3. ADD vs COPY Comparison

Feature	ADD	COPY
Copies local files	Yes	Yes
Extracts local tar files	Yes (automatic)	No
Supports remote URLs	Yes (HTTP/HTTPS)	No
More explicit behaviour	No (can be implicit)	Yes
Recommended for application files	No	Yes
Use case	Tar extraction, remote files, advanced scenarios	Regular application files and directories

4. Examples of Using ADD

Example 1: Add a local tar file (automatic extraction)

```
# Dockerfile
FROM ubuntu:22.04
ADD app.tar.gz /opt/app/
# app.tar.gz is automatically extracted to /opt/app/
```

Example 2: Add a remote file from a URL

```
# Dockerfile
FROM ubuntu:22.04
ADD https://example.com/tool.tar.gz /usr/local/
# Downloads the file and automatically extracts it
```

Example 3: Add a single local file

```
# Dockerfile
FROM python:3.12-slim
ADD config.json /app/config.json
# Works like COPY for regular files
```

5. Examples of Using COPY

Copy application files

```
# Dockerfile
FROM python:3.12-slim
COPY . /app
# Copies files from build
# context to /app
```

Copy specific files

```
# Dockerfile
FROM python:3.12-slim
COPY requirements.txt /app/
COPY src/ /app/src/
```

6. Real-World Use Cases for ADD

- ✓ Adding a prebuilt application package (tar.gz) that should be extracted inside the image.
- ✓ Downloading and extracting a tool from a trusted URL.
- ✓ Adding compressed dependencies or assets.
- ✓ Quickly including third-party binaries or SDKs.

7. When to Use COPY Instead

- ✗ Copying your application source code.
- ✗ Copying configuration files, scripts, or static assets.
- ✗ When you do not need tar extraction or remote download.
- ✗ When you want clear and predictable behaviour.
- ✗ To avoid accidentally downloading files from untrusted URLs.

8. Example Project Structure and Usage

```
myapp/
├── app.tar.gz      # prebuilt app package
├── config.json     # configuration file
├── requirements.txt
└── src/
    └── app.py
```

```
# Dockerfile
FROM python:3.12-slim
# Use ADD to extract tar file
ADD app.tar.gz /opt/app/
# Use COPY for app files
COPY requirements.txt /app/
COPY src/ /app/src/
```

9. Key Takeaways

1. ADD can copy files, extract tar archives, and download remote files.
2. COPY only copies local files from the build context.
3. COPY is usually preferred for application files.
4. Use ADD only when you need tar extraction or remote URLs.
5. Be careful with remote sources and always use trusted URLs.
6. Be explicit to avoid unexpected behaviour in your builds.

VERIQTA

HEALTHCHECK: Teaching Docker How to Check Container Health

1. Running vs Healthy

- A container can be running even if the application inside is not working correctly.
- A running process only means the main process has not exited.
- A healthy container means the application is responding as expected (e.g., HTTP endpoint, database connection, or required service is working).
- HEALTHCHECK tells Docker how to check the health of your application.

2. Basic HEALTHCHECK Example

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
RUN pip install flask
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:8000/ || exit 1
CMD ["python", "app.py"]
```

3. HEALTHCHECK Options Explained

Option	Description	Example
--interval	How often to run the check (default: 30s).	--interval=30s
--timeout	How long to wait for the check to complete (default: 30s).	--timeout=5s
--retries	Number of consecutive failures before marked unhealthy (default: 3).	--retries=3
--start-period	Time to ignore failures while the application starts (default: 0s).	--start-period=20s
CMD	Command to run for the health check. Exit code 0 = healthy, non-zero = unhealthy.	CMD curl -f http://localhost:8000/

4. Health Check Exit Status

Exit Code	Meaning
0	Healthy (check passed)
1	Unhealthy (check failed)
2	Reserved for future use

5. Inspecting Container Health

```
# Check container status
docker ps
CONTAINER ID   IMAGE   COMMAND           STATUS
a1b2c3d4e5f6   myapp   "python app.py"   Up (healthy)
```

```
# View detailed health information
docker inspect --format='{{json .State.Health}}' myapp
{
  "Status": "healthy",
  "FailingStreak": 0,
  "Log": [
    {
      "Start": "2026-10-14T12:00:00Z",
      "End": "2026-10-14T12:00:01Z",
      "ExitCode": 0,
      "Output": "HTTP/1.1 200 OK"
    }
  ]
}
```

6. Different Ways to Write HEALTHCHECK

Using curl (HTTP check)

```
HEALTHCHECK --interval=30s \
  --timeout=5s --retries=3 \
 CMD curl -f http://localhost:8000/ \
  || exit 1
```

Using wget

```
HEALTHCHECK --interval=30s \
  --timeout=5s --retries=3 \
 CMD wget -qO- http://localhost:8000/
  || exit 1
```

Using a script

```
COPY healthcheck.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/healthcheck.sh
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD /usr/local/bin/healthcheck.sh
```

Checking a TCP port

```
HEALTHCHECK --interval=30s \
  --timeout=5s --retries=3 \
 CMD bash -c "</dev/tcp/localhost/8000" \
  || exit 1
```

7. Important Things to Know

- A failed health check does NOT automatically restart a standalone Docker container.
- Health checks only report the status (healthy or unhealthy).
- Restart behaviour must be handled by an orchestrator (e.g., Docker Compose, Kubernetes) or a custom restart policy.
- Use a meaningful health check that tests the actual application (e.g., HTTP endpoint, database connection, or required service).
- Give the application enough time to start using --start-period.

8. Example: Unhealthy Container

```
$ docker ps
CONTAINER ID   IMAGE   COMMAND           STATUS
f6e5d4c3b2a1   myapp   "python app.py"   Up (unhealthy)
```

9. Key Takeaways

1. HEALTHCHECK checks your application, not just the process.
2. Use appropriate interval, timeout, and retries.
3. Exit code 0 means healthy, non-zero means unhealthy.
4. Inspect health status with docker ps and docker inspect.
5. A failed health check does not automatically restart a container.
6. Use orchestrators for automatic recovery and restarts.

VERIQTA

SHELL, STOPSIGNAL, LABEL, VOLUME, and ONBUILD

1. SHELL: Set the Default Shell for RUN

- **SHELL** changes the default shell used to run subsequent **RUN** commands.
- By default, Linux images use ["/bin/sh", "-c"] and Windows images use ["cmd", "/S", "/C"].
- You can change the shell to use bash with useful options (e.g., **-eo pipefail**).
- Helps with consistent behaviour and error handling in complex commands.

Example: Use bash as the default shell

```
# Dockerfile
FROM ubuntu:22.04
SHELL ["/bin/bash", "-eo", "pipefail", "-c"]
RUN echo "Using bash with pipefail" && \
    echo "If any command fails, the build fails"
```

When to use:

- When you need bash features (arrays, pipefail).
- For better error handling in multi-command RUN.
- When running complex scripts.

2. STOPSIGNAL: Define the Container Stop Signal

- **STOPSIGNAL** sets the system call signal that Docker sends when you run **docker stop**.
- Default is **SIGTERM (15)** on Linux.
- You can use a different signal if your application handles another signal better (e.g., **SIGQUIT**).

Example: Use SIGQUIT instead of default SIGTERM

```
# Dockerfile
FROM nginx:1.25
STOPSIGNAL SIGQUIT
CMD ["nginx", "-g", "daemon off;"]
```

When to use:

- When your application handles a specific signal for graceful shutdown.
- Common for Nginx (SIGQUIT) and some Java applications.

3. LABEL: Add Metadata to Your Image

- **LABEL** adds key-value metadata to an image.
- Useful for documenting the image, ownership, version, source repo, and more.
- Helps with image management, scanning, and organization.

Example: Add useful labels

```
# Dockerfile
FROM python:3.12-slim
LABEL org.opencontainers.image.title="myapp"
LABEL org.opencontainers.image.description="A sample Python application"
LABEL org.opencontainers.image.version="1.0.0"
LABEL org.opencontainers.image.source="https://github.com/veriqta/myapp"
LABEL maintainer="veriqta@example.com"
```

When to use:

- Always a good practice for production images.
- Useful for tracking, security scanning, and automated tools.
- Follow standard label keys (e.g., org.opencontainers.image.*).

4. VOLUME: Declare Mount Points

- **VOLUME** declares a directory as a mount point for external storage.
- Data written to this directory can be persisted using a Docker volume or bind mount.
- Useful for application data, logs, databases, or uploads.

Example: Declare a volume

```
# Dockerfile
FROM nginx:1.25
VOLUME ["/var/log/nginx"]
VOLUME ["/usr/share/nginx/html"]
```

When to use:

- When data needs to persist outside the container.
- For logs, uploads, or database storage.
- Use named volumes or bind mounts at runtime.

Using the volume at runtime

```
# Create a named volume and run the container
docker volume create nginx-logs
docker run -d --name web \
  -p 8080:80 \
  -v nginx-logs:/var/log/nginx \
  -v $(pwd)/site:/usr/share/nginx/html
    nginx:1.25
```

5. ONBUILD: Trigger Instructions in Child Images

- **ONBUILD** adds a trigger instruction that is executed when another image uses this image as a base.
- It does not run in the current build. It runs only in child builds.
- Commonly used in base images to define standard behaviour for application images.
- Can be powerful but may be confusing, so use carefully.

Example: Base image with ONBUILD

```
# Dockerfile (base image)
FROM python:3.12-slim
WORKDIR /app
ONBUILD COPY requirements.txt .
ONBUILD RUN pip install -r requirements.txt
ONBUILD COPY . .
ONBUILD CMD ["python", "app.py"]
```

When to use:

- When creating a base image for multiple applications.
- Useful in organization standard images.
- Use carefully, as it can make builds less transparent.

6. Important Considerations

VOLUME Considerations

- Data is not stored in the image itself.
- Can make containers less portable if not managed.
- Use named volumes or bind mounts for control.
- Be careful with permissions and ownership.
- Not ideal for application configuration files (use COPY or bind mounts instead).

ONBUILD Considerations

- Instructions are triggered in child images, not in the current build.
- Can be surprising for users who do not expect inherited steps.
- Harder to debug when builds fail.
- Should be clearly documented in the base image.
- Avoid overusing it, as it can reduce flexibility for downstream users.

7. Key Takeaways

1. **SHELL** sets the default shell for RUN commands.
2. **STOPSIGNAL** defines which signal Docker sends on container stop.
3. **LABEL** adds useful metadata to an image.
4. **VOLUME** declares a mount point for external storage.
5. **ONBUILD** triggers instructions in child image builds.
6. Use these instructions intentionally and understand their impact on container behaviour and downstream users.

VERIQTA

Multi-Stage Builds: Separating Build Tools From Runtime

1. What Are Multi-Stage Builds?

- Multi-stage builds use multiple **FROM** instructions in the same Dockerfile.
- Each stage has its own filesystem.
- You can name stages and copy only the files you need from a previous stage using **COPY --from**.
- This allows you to keep build tools, compilers, and temporary files out of the final image.
- The result is a smaller, cleaner image with fewer unnecessary files.

2. How It Works (Overview)

- Stage **1 (builder)**: uses a full image with build tools to install dependencies and prepare the application.
- **Stage 2 (runtime)**: uses a smaller image and only copies the required artifacts from the builder stage.
- Only the files you explicitly copy are included in the final image.
- Everything else in the builder stage (build tools, caches, source files, etc.) stays behind.

Builder Stage
(FROM python:3.12-slim)
- Build tools
- Dependencies
- Source code
- Temporary files

COPY --from=builder →

Runtime Stage
(FROM python:3.12-slim)
- Only what you copy
- Application files
- Runtime dependencies
- No build tools

3. Example: Python Application with Multi-Stage Build

```
# Stage 1: Build the application and install dependencies
FROM python:3.12-slim AS builder
WORKDIR /app

# Install build dependencies (if needed)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copy only requirements file and install Python packages
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Copy application source code
COPY . .

# Stage 2: Create a smaller runtime image
FROM python:3.12-slim AS runtime
WORKDIR /app

# Copy installed Python packages from builder stage
COPY --from=builder /root/.local /root/.local

# Copy application code from builder stage
COPY --from=builder /app /app

# Set environment variables
ENV PATH=/root/.local/bin:$PATH
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Run the application
CMD ["python", "app.py"]
```

4. What Actually Gets Copied?

Item	Copied to Runtime?	Explanation
/root/.local (installed packages)	✅ Yes	Explicitly copied using COPY --from=builder
/app (application code)	✅ Yes	Explicitly copied using COPY --from=builder
Build tools (gcc, make, etc.)	❌ No	Remains only in builder stage
Apt cache and temp files	❌ No	Remains only in builder stage
Source files not copied	❌ No	Only files you copy are included
Everything else from builder	❌ No	Stays behind in the builder stage

5. Build and Run the Image

```
# Build the image
docker build -t myapp:multi .

# Check image size (should be smaller than single-stage)
docker images myapp:multi

# Run the container
docker run --rm -p 8000:8000 myapp:multi
```

6. Compare Image Sizes

Build Type	Image Size (Example)	Why
Single-stage (with build tools)	450 MB	Includes Python, build tools, apt cache, and source code.
Multi-stage (runtime only)	120 MB	Only includes runtime, installed packages, and application code.

7. Important Notes

- ✅ Multi-stage builds reduce image size and remove unnecessary files.
- ✅ They help you keep build tools and sensitive files out of the final image.
- ✅ They do NOT automatically make your image secure.
- ✅ You still need to use a minimal base image, keep your dependencies up to date, and follow security best practices.
- ✅ Always copy only what you need from the builder stage.

8. When to Use Multi-Stage Builds

- ✅ When your application needs build tools or compilers.
- ✅ When you want a smaller and cleaner runtime image.
- ✅ When you want to separate build-time and runtime concerns.
- ✅ Common for compiled languages (Go, Rust, C/C++) and also useful for Python apps.

9. Key Takeaways

1. Use multiple **FROM** instructions to create build and runtime stages.
2. Use **COPY --from** to copy only the required files.
3. Build tools and temporary files stay behind in the builder stage.
4. The final image is smaller and cleaner.
5. Multi-stage builds improve image quality but do not guarantee security.

VERIQTA

Debugging Dockerfile Failures Like an Engineer

1. Use Detailed Build Output

- By default, Docker hides some output.
- Use `--progress=plain` to see full logs and find the real error.
- Look at the exact step that failed and read the full error message.

Build with detailed output

```
docker build --progress=plain -t myapp:debug .
# Shows complete output for each step
# Helps you see the exact error and line number
```

2. Missing Build-Context Files

- Docker can only access files inside the build context (the folder you run `docker build` from).
- If a file is missing, `COPY` or `ADD` will fail.

Example Error

```
COPY requirements.txt /app/
ERROR: failed to compute cache key:
failed to calculate checksum: "/requirements.txt": not found
```

Fix It

- ✓ Make sure the file exists in your project folder.
- ✓ Use the correct path relative to the build context.
- ✓ Check your `.dockerignore` is not excluding the file.

3. COPY Path Errors

- The source path in `COPY` must exist in the build context.
- Incorrect paths or typos will cause build failures.
- Destination directories may need to be created first.

Example Error

```
COPY app.py /app/
ERROR: failed to compute cache key:
"app.py": not found
```

Fix It

- ✓ Check file name and path.
- ✓ Use correct relative paths.
- ✓ Create destination directory. if needed (e.g., RUN mkdir -p /app).

4. Dependency Installation Failures

- Network issues, incorrect package names, or missing system dependencies can cause installation failures.
- Always read the full error message.
- You may need additional OS packages for some Python packages.

Example Error

```
RUN pip install -r requirements.txt
ERROR: Could not find a version
that satisfies the requirement
somepackage==1.0.0
```

Fix It

- ✓ Verify package names and versions.
- ✓ Check internet access.
- ✓ Install required OS packages (e.g., build-essential).

5. Permission Denied

- Can happen when writing to restricted directories or running as a non-root user without proper permissions.

Example Error

```
RUN pip install -r requirements.txt
ERROR: [Errno 13] Permission denied:
'/usr/local/lib/python3.12'
```

Fix It

- ✓ Use a user with the right permissions.
- ✓ Install to a writable location or use `--user`.
- ✓ Change ownership with chown if needed.

6. Missing Executable or Command Not Found

- The command or script you try to run may not exist or may not be executable.

Example Error

```
CMD ["python", "app.py"]
python: can't open file 'app.py':
No such file or directory
```

Fix It

- ✓ Ensure the file is copied to the correct location.
- ✓ Use the correct command path.
- ✓ Make sure the file has execute permissions if needed (chmod +x script.sh).

7. Incorrect Command Form

- The wrong `CMD` or `ENTRYPOINT` form can cause the container to exit immediately or not find the command.

Problem (shell form)

```
CMD python app.py
```

- ✗ Uses a shell, which can hide errors.
- ✗ Signal handling may not work properly.
- ✗ Less predictable behaviour.

Recommended (exec form)

```
CMD ["python", "app.py"]
```

- ✓ Runs the command directly.
- ✓ Better signal handling.
- ✓ More predictable and preferred.

8. Port Mismatch

- The application may be listening on a different port than expected, or the port may not be published when running the container.

Example

```
# App listens on port 8000
EXPOSE 8000
# But you run the container with the
# wrong port mapping
docker run -p 8080:80 myapp
```

- ✗ Container runs, but the application is not reachable.

Fix It

- ✓ Verify which port your application listens on.
- ✓ Use the correct port mapping: docker run -p 8080:8000 myapp
- ✓ Check application logs to confirm it started on the expected port.

9. Application Startup Failure

- The container may start but the application exits due to misconfiguration, missing environment variables, or runtime errors.
- Use logs to find the real cause.

Check container logs

```
docker logs myapp
# Shows application output and errors
```

Check container details

```
docker inspect myapp
# View exit code, configuration,
# and other details
```

10. Step-by-Step Debugging Process

1. Read the full error message (use `--progress=plain`).
2. Check that all files exist in the build context.
3. Verify COPY paths and destination directories.
4. Check dependency installation errors and required OS packages.
5. Ensure correct permissions and users.
6. Verify the command form (prefer exec form).
7. Confirm the application port and run with the correct mapping.
8. Check container logs and inspect for more details.
9. Fix one issue at a time and rebuild.

VERIQTA

Final Dockerfile: From First Build to Production-Aware Image

1. What This Final Image Includes

- Multi-stage build (separates build tools from runtime)
- Pinned dependencies for reproducibility
- Suitable base image (lightweight and stable)
- Build caching (copies dependency files first)
- Non-root user for security
- Environment configuration
- Documented port
- Health check for application readiness
- Clear startup command
- Smaller, cleaner, production-aware image

2. Complete Multi-Stage Dockerfile (Python Example)

```
FROM python:3.12-slim AS builder              # Build stage with tools
WORKDIR /app                                  # Set working directory
COPY requirements.txt .                       # Copy dependencies first
RUN pip install --no-cache-dir -r requirements.txt  # Install Python packages

FROM python:3.12-slim AS runtime              # Final runtime stage
RUN useradd -m -u 10001 appuser               # Create non-root user
WORKDIR /app                                  # Set working directory
COPY --from=builder /usr/local/lib/python3.12/  # Copy installed packages
     /usr/local/lib/python3.12/               # from builder stage
COPY --from=builder /app .                    # Copy application code
ENV PYTHONDONTWRITEBYTECODE=1                 # Prevent .pyc files
ENV PYTHONUNBUFFERED=1                        # Better logging
ENV APP_ENV=production                        # Application environment
EXPOSE 8000                                   # Document application port
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
     CMD curl -f http://localhost:8000/ || exit 1   # Check if app is healthy
USER appuser                                  # Run as non-root user
CMD ["python", "app.py"]                      # Start the application
```

3. Dockerfile Line-by-Line Explanation

Line(s)	Explanation
1	Use an official slim Python image for the builder stage.
2	Set the working directory inside the container.
3	Copy only requirements.txt to leverage build cache.
4	Install pinned dependencies without caching packages.
6	Start a new, clean runtime stage (no build tools).
7	Create a non-root user for better security.
8	Set the working directory.
9	Copy installed Python packages from the builder stage.
10	Copy the application source code.
11–13	Set environment variables for runtime behaviour.
14	Document the application port.
15–16	Add a health check to verify the application is running.
17	Switch to the non-root user.
18	Define the startup command using exec form.

4. Project Structure

```
myapp/
├── Dockerfile
├── requirements.txt
└── app.py
```

Sample requirements.txt

```
flask==3.0.3
gunicorn==22.0.0
```

Sample app.py (simplified)

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from VERIQTA!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

5. Build, Run, Test, Inspect, Stop and Cleanup

Build the image

```
docker build -t myapp:1.0 .
```

Run the container

```
docker run --rm -p 8000:8000 --name myapp myapp:1.0
```

Test the application

```
curl http://localhost:8000/
```

Check container health

```
docker inspect --format='{{json .State.Health}}' myapp
```

View application logs

```
docker logs myapp
```

Stop the container (if not using --rm)

```
docker stop myapp
```

Cleanup (remove container and image)

```
docker rm myapp
docker rmi myapp:1.0
```

6. Security and Operational Checklist

- Use official base images and keep them updated.
- Pin dependency versions.
- Run as a non-root user.
- Use multi-stage builds to keep the final image small.
- Avoid copying unnecessary files (use .dockerignore).
- Set only the required environment variables.
- Include a meaningful health check.
- Scan the image for vulnerabilities (e.g., docker scout, Trivy).
- Log to stdout/stderr and avoid writing logs inside the container.
- Regularly rebuild and test the image.

7. Key Takeaways

1. Multi-stage builds create smaller and cleaner images.
2. Pinned dependencies improve reproducibility and reduce risk.
3. Copy dependency files first to take advantage of build cache.
4. Run containers as a non-root user for better security.
5. Environment variables control runtime behaviour.
6. EXPOSE documents the port; use -p to publish it.
7. HEALTHCHECK helps you detect when the application is ready.
8. The CMD should be clear and use the exec form.
9. Always test, inspect, and verify the container before using it in production.
10. A well-designed Dockerfile improves security, performance, and maintainability.